MQTT Firewall Intrusion Detection Report

Firewall System Overview

Architecture and Execution Model

The MQTT firewall operates as an inline, flow-aware intrusion detection and prevention system purpose-built for IoT broker traffic. Rather than relying on signature matching or rule-based filtering, it applies a trained machine learning model to classify network flows in real time, making decisions at the granularity of individual TCP connections rather than individual packets.

The system is designed to run as a persistent background service (managed via systemd) that intercepts all traffic destined for or originating from the MQTT broker on port 1883. It operates in one of two modes depending on configuration:

- **IDS mode (Intrusion Detection System):** All traffic passes through unmodified. Detected attacks are logged with full metadata — source, attack type, and model confidence — but no connection is blocked. This mode is suited for passive monitoring and research evaluation.
- **IPS mode (Intrusion Prevention System):** On top of detection, the source IP of any confirmed attack is added to an internal blocklist, and subsequent packets from that address are immediately discarded. An optional TTL-based auto-unblock mechanism can be configured to lift blocks after a specified duration, preventing indefinite denial of service to reconnecting devices.

Packet Interception via NetfilterQueue

At the kernel level, Linux `iptables` rules redirect all TCP packets on port 1883 — both inbound (`INPUT`) and outbound (`OUTPUT`) — into a **NetfilterQueue (NFQUEUE)**. This is a Linux kernel hook that holds each intercepted packet in a queue and hands it to a user-space process before forwarding or dropping it.

The firewall process binds to this queue and receives packets one at a time through a callback function (`handle_packet`). Each packet is parsed using the `dpkt` library, which decodes the raw IP and TCP headers to extract the following fields:

- Source and destination IP addresses
- Source and destination TCP ports
- Protocol number (TCP = 6)

- Payload length and TCP header length
- TCP flags (SYN, ACK, PSH, FIN, RST, ECE, CWR)

After parsing, every packet is **immediately accepted** back to the kernel — ensuring no additional latency is introduced to MQTT communication — before any classification logic runs. This passive acceptance model means the firewall's analytical work never blocks or delays legitimate broker traffic.

Flow Table and State Tracking

Individual packets carry limited information on their own. To build meaningful network behaviour profiles, the firewall maintains a **bidirectional flow table** in memory. Each entry in this table represents a unique TCP connection, identified by the five-tuple `(src_ip, dst_ip, src_port, dst_port, protocol)`.

As each packet arrives, it is associated with its corresponding flow entry (creating a new one if none exists). The flow record accumulates:

- Forward and backward packet counts and byte totals
- Per-direction packet length lists
- Per-direction inter-arrival times (IAT)
- Cumulative TCP flag counts across all packets in the flow

This state is maintained across the lifetime of a connection, enabling the extraction of statistical behavioural features that span multiple packets — a necessity for reliably distinguishing attack patterns from normal broker traffic.

Classification Triggers

Classification is triggered under two distinct conditions, each suited to a different attack profile:

1. Connection termination (FIN / RST): When a TCP FIN or RST flag is observed, the flow is considered complete. If it has accumulated at least the configured minimum number of packets (`FW_MIN_PKTS_CLASSIFY`, default: 3), the feature vector is extracted and passed to the model immediately. This path captures short-lived attack behaviours such as port scans and TCP-level probes that open and close connections rapidly.

2. Periodic classification of persistent connections: Many MQTT attack patterns — such as publish floods and connect floods — maintain long-lived TCP connections that never send a FIN or RST. To handle these, the firewall classifies any active flow every `FW_CLASSIFY_EVERY_N_PKTS` packets (default: 5). After classification, the flow entry is removed and a fresh one is started, allowing each

classification window to reflect only recent traffic behaviour rather than accumulating a biased long-term average.

A background **reaper thread** runs every second in parallel, scanning the flow table for entries that have been idle for longer than `FW_FLOW_TIMEOUT_S` seconds (default: 10). Any idle flow with sufficient packet count is classified and removed, ensuring that connections which simply go quiet are not left in an unclassified state indefinitely.

Machine Learning Model and Preprocessing

The classification engine is a **LightGBM gradient-boosted decision tree model** trained on the CIC IoMT 2024 dataset — a publicly available benchmark dataset of MQTT and network attack traffic collected in controlled IoT environments. The model is a multi-class classifier covering 21 distinct traffic categories, including various flooding, reconnaissance, and protocol-abuse attack types, as well as a benign class representing normal MQTT communication.

Before training, all numeric features were normalised using a **StandardScaler** (zero mean, unit variance). The fitted scaler is serialised as a `.pkl` file (`standard_scaler.pkl`) and loaded at firewall startup. At inference time, every extracted feature vector is passed through this scaler before being submitted to the model, ensuring the numerical range seen by the model at runtime matches what it encountered during training.

The model itself is stored in LightGBM's native text format (`lightgbm_model.txt`). A **LabelEncoder** (also serialised as `label_encoder.pkl`) maps the model's integer output indices to human-readable class names such as `MQTT-DDoS-Connect_Flood`, `Recon-Port_Scan`, or `Benign`. These class names are also embedded in the runtime configuration via the `FW_CLASS_NAMES` environment variable, allowing the firewall to produce labelled verdicts without loading the encoder at runtime.

Feature Extraction (45-Dimensional Vector)

Once a classification trigger fires, the flow record is converted into a 45-dimensional numerical feature vector that exactly mirrors the column layout of the CIC IoMT 2024 training dataset. The features fall into the following groups:

Group	Features
Timing & rate	Flow duration, total packet rate, forward rate (Srate), backward rate (Drate)
TCP flags	Per-flow counts of FIN, SYN, RST, PSH, ACK, ECE, CWR flags

Group	Features
Protocol indicators	Binary flags for HTTP, HTTPS, DNS, SSH, TCP, UDP, ICMP, and others
Packet size statistics	Total bytes, minimum, maximum, mean, and standard deviation of packet lengths
Inter-arrival time	Mean IAT across all packets in the flow
Derived statistical features	Magnitude (RMS of directional means), Radius (RMS of directional variances), Covariance (fwd ↔ bwd cross-covariance), Variance (ratio of directional variances), Weight (product of forward and backward packet counts — near zero for unidirectional attack flows)
Header information	Mean TCP/IP header length across the flow

The **Weight** feature is particularly discriminative for flood attacks: legitimate MQTT publish cycles generate symmetric bidirectional traffic (publish → ACK), whereas a pure publish flood generates a heavily asymmetric flow where `fwd_pkts >> bwd_pkts`, driving Weight toward zero.

Verdict and Response

Once the model produces a probability vector over all 21 classes, the predicted class is the one with the highest probability. A flow is labelled **ATTACK** if the predicted class is not the configured benign class and the probability assigned to the benign class falls below `1 − FW_ATTACK_THRESHOLD` (default threshold: 0.5). This dual condition prevents low-confidence edge cases from being incorrectly blocked.

Every verdict — whether attack or benign — is recorded in two complementary output files:

- **`mqtt_firewall.log`** — a human-readable, line-oriented log containing timestamps, flow identifiers, verdict labels, attack type names, and confidence scores.
- **`mqtt_firewall.jsonl`** — a machine-readable JSON Lines sidecar containing the full event payload for every packet parse, pre-inference feature vector, and verdict, suitable for offline analysis and report generation.

In IPS mode, a confirmed attack verdict additionally causes the source IP to be registered in an in-memory blocklist. All subsequent packets from that IP are accepted at the kernel level but bypassed entirely in the analysis pipeline, reducing CPU overhead during sustained attack campaigns.

---

1. Executive Summary

The LightGBM-based MQTT firewall processed **26,611 packets** (31,947 log lines) and issued **5,329 flow-level verdicts**. Of these, **5,309 (99.6%) were classified as attacks** and **20 (0.4%) as benign**. Mean model confidence on attack flows was **1.0000**.

Metric	Value
Total log lines	31,947
Total packets intercepted	26,611
Total flow verdicts	5,329
Attack verdicts	5,309 (99.6%)
Benign verdicts	20 (0.4%)
Observation window	02:02:18 → 02:03:33

2. Attack Type Breakdown

Attack Type	Verdict Count	Share (%)
MQTT-DDoS-Connect_Flood	4,844	90.9%
MQTT-DoS-Publish_Flood	283	5.3%
MQTT-DDoS-Publish_Flood	115	2.2%
Recon-Port_Scan	34	0.6%
MQTT-DoS-Connect_Flood	32	0.6%
benign	20	0.4%
TCP_IP-DDoS-TCP	1	0.0%

3. Model Confidence Statistics

Statistics computed over **attack-classified flows only**.

Statistic	Value
Sample size	5,309 flows
Minimum confidence	0.8924
Maximum confidence	1.0000
Mean confidence	1.0000
Flows with confidence ≥ 0.90	5,308 (100.0%)

4. Flow Statistics

Metric	Min	Max	Mean
Packets per flow	3	5	5.0
Bytes per flow	0	19,820	1,704
Flow duration (s)	-1.8357	15.0088	0.1096

5. Top Attacker Flows

48 distinct `(src_ip:src_port, attack_type)` flows detected.

Source (ip:port)	Classified Attack Type	Verdict Count
127.0.0.1:1883	MQTT-DDoS-Connect_Flood	2,469
127.0.0.1:60255	MQTT-DDoS-Connect_Flood	2,362
127.0.0.1:1883	MQTT-DoS-Publish_Flood	91
127.0.0.1:43655	MQTT-DoS-Publish_Flood	80
127.0.0.1:48919	MQTT-DoS-Publish_Flood	80
127.0.0.1:1883	MQTT-DDoS-Publish_Flood	56
127.0.0.1:60255	MQTT-DoS-Publish_Flood	32
127.0.0.1:43655	MQTT-DDoS-Publish_Flood	31
127.0.0.1:1883	MQTT-DoS-Connect_Flood	30
127.0.0.1:48919	MQTT-DDoS-Publish_Flood	28
127.0.0.1:43655	MQTT-DDoS-Connect_Flood	10
127.0.0.1:48919	MQTT-DDoS-Connect_Flood	3
127.0.0.1:60255	Recon-Port_Scan	2
127.0.0.1:43655	Recon-Port_Scan	1
127.0.0.1:48919	Recon-Port_Scan	1
127.0.0.1:48374	Recon-Port_Scan	1
127.0.0.1:48386	Recon-Port_Scan	1
127.0.0.1:48402	Recon-Port_Scan	1
127.0.0.1:48418	Recon-Port_Scan	1
127.0.0.1:48420	Recon-Port_Scan	1

6. Last 15 Verdicts

Time	Verdict	Attack Type	Confidence	Source	Destination	Pkts	Byte
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:1883	127.0.0.1:60255	5	990
02:03:22	ATTACK	MQTT-DoS-Publish_Flood	1.0000	127.0.0.1:1883	127.0.0.1:43655	5	4,128
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:60255	127.0.0.1:1883	5	1,485
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:1883	127.0.0.1:60255	5	990
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:60255	127.0.0.1:1883	5	1,485
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:1883	127.0.0.1:60255	5	990
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:60255	127.0.0.1:1883	5	1,485
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:1883	127.0.0.1:60255	5	990
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:60255	127.0.0.1:1883	5	1,485
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:1883	127.0.0.1:60255	5	990
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:60255	127.0.0.1:1883	5	1,485
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:1883	127.0.0.1:60255	5	990
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:60255	127.0.0.1:1883	5	1,485
02:03:22	ATTACK	MQTT-DDoS-Connect_Flood	1.0000	127.0.0.1:1883	127.0.0.1:60255	5	990
02:03:22	ATTACK	Recon-Port_Scan	1.0000	127.0.0.1:60255	127.0.0.1:1883	3	495
02:03:33	ATTACK	MQTT-DoS-Connect_Flood	1.0000	127.0.0.1:43655	127.0.0.1:1883	3	0
02:03:33	ATTACK	MQTT-DoS-Connect_Flood	1.0000	127.0.0.1:48919	127.0.0.1:1883	3	0